

IRONCORE GYM

Guía de estudio: cómo está hecha la web — HTML semántico, CSS (Flexbox, Grid, box model, animaciones, responsive) y accesibilidad, explicados con el código real del proyecto.

Proyecto: sitio web de 5 vistas para un gimnasio
Tecnologías: HTML5 + CSS3 puro (sin JavaScript)
Carpeta: ED/WebGimnasio

1. INTRODUCCIÓN Y MAPA DEL SITIO

El proyecto está formado por **5 páginas HTML** que comparten un mismo encabezado, una misma navegación y una única hoja de estilos (`estilos.css`). Ningún archivo usa estilos internos (`<style>`) ni en línea (`style="..."`), y no hay ningún archivo `.js`: todo el comportamiento interactivo (menú móvil, tema claro/oscuro) está resuelto solo con HTML y CSS. Es la misma arquitectura que el proyecto "Café Aroma", pero reskinada por completo para un gimnasio.

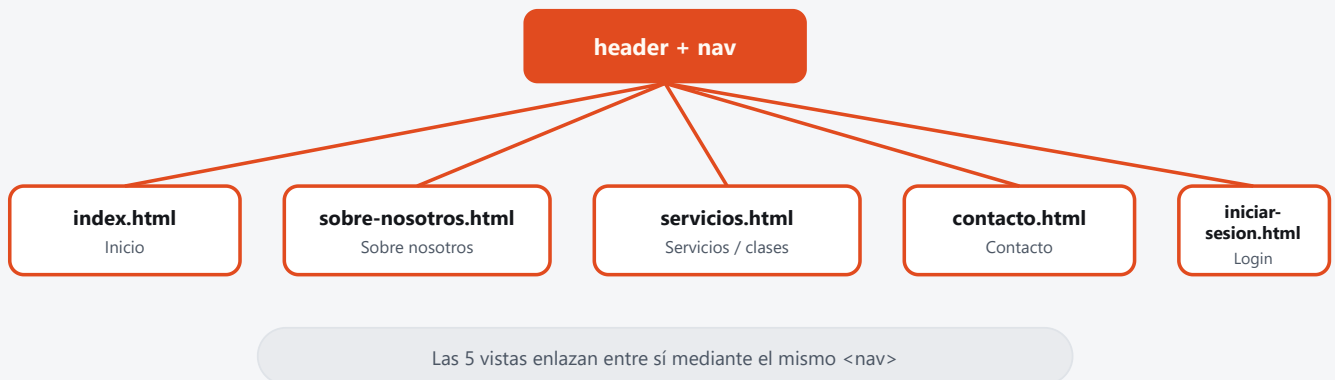


Fig. 1 — Las 5 vistas comparten un mismo header/nav construido en Flexbox y se enlazan entre sí.

ÁRBOL DE ARCHIVOS DEL PROYECTO

```
WebGimnasio/
├─ index.html           → Vista 1: Inicio
├─ sobre-nosotros.html  → Vista 2: Sobre nosotros
├─ servicios.html       → Vista 3: Servicios / clases
├─ contacto.html        → Vista 4: Contacto
├─ iniciar-sesion.html   → Vista 5: Iniciar sesión
├─ estilos.css          → única hoja de estilos (compartida)
├─ assets/              → ilustraciones SVG propias
│   └─ favicon.svg, hero-fitness.svg, historia.svg
│   └─ musculacion.svg, funcional.svg, yoga.svg, spinning.svg
```

2. ESTRUCTURA SEMÁNTICA HTML

En vez de construir todo con `<div>`, cada vista usa etiquetas que describen el *rol* del contenido. Esto ayuda a los lectores de pantalla y a los buscadores a entender la página sin necesidad de leer el CSS.

```
<body>
  <header class="site-header">           // se repite igual en las 5 vistas
    <nav aria-label="Navegación principal">...</nav>
  </header>

  <main>                                 // un solo <main> por página
    <section class="section">
      <h1>...</h1>
      <article>...</article>           // bloques de contenido independiente
    </section>
  </main>

  <footer class="site-footer">...</footer>
</body>
```

Esqueleto repetido en `index.html`, `sobre-nosotros.html`, `servicios.html`, `contacto.html` e `iniciar-sesion.html`.

POR QUÉ IMPORTA

`<header>`, `<nav>`, `<main>`, `<footer>`, `<section>` y `<article>` no cambian nada visualmente por sí solos (son como `<div>` a efectos de estilo), pero le dicen al navegador y a las tecnologías de asistencia **qué es cada parte**: "esto es la cabecera", "esto es el contenido principal", "esto es un bloque autocontenido". Es la base de la accesibilidad y del SEO.

3. METAETIQUETAS BÁSICAS DE SEO

En el <head> de cada vista hay un bloque de metaetiquetas que describe la página para buscadores y navegadores, con contenido distinto en cada una (no se copia el mismo texto 5 veces):

```
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1.0" />
<title>IronCore Gym – Entrena con propósito</title>
<meta name="description" content="IronCore Gym es un gimnasio de entrenamiento funcional..." />
<meta name="keywords" content="gimnasio, fitness, musculación..." />
<meta name="author" content="IronCore Gym" />
```

Etiqueta	Para qué sirve
charset	Define la codificación de caracteres (para que tildes y "ñ" se vean bien).
viewport	Le dice al navegador móvil que use el ancho real del dispositivo. Es la base del diseño responsivo.
title	Texto de la pestaña del navegador y título en resultados de búsqueda.
description	Resumen que Google suele mostrar debajo del enlace en los resultados.
keywords	Lista de temas relacionados con la página (peso bajo hoy en SEO, pero sigue siendo buena práctica declararlo).

4. EL MODELO DE CAJA (BOX MODEL)

Todo elemento HTML es, para efectos de CSS, una caja rectangular con cuatro capas. El formulario de contacto.html es un buen ejemplo: cada campo usa *padding* para que el texto no toque el borde, *border* para delimitarlo y *margin* para separarlo del siguiente campo.

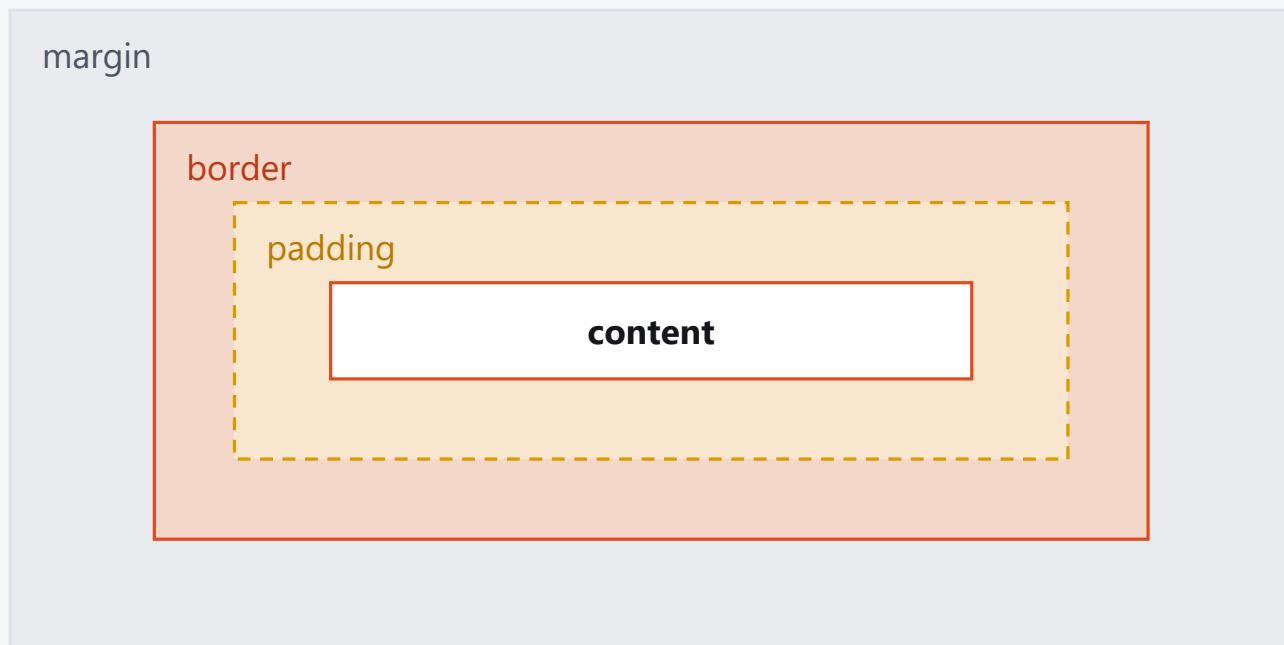


Fig. 2 — Modelo de caja: content → padding → border → margin.

```
.form-group input,  
.form-group select,  
.form-group textarea {  
  padding: 0.7rem 0.9rem; /* espacio interno → padding */  
  border: 1px solid var(--color-border);  
  border-radius: 8px;  
}  
  
.form-group {  
  margin-bottom: 1.25rem; /* separación entre campos → margin */  
}
```

De `estilos.css`, sección "Formularios".

5. FLEXBOX — HEADER Y NAV

El requisito pide que el header y el nav se construyan con **Flexbox**. En `.site-header .container` se activa `display: flex` para alinear el logo, el menú y el botón "Iniciar sesión" en una fila, repartidos con `justify-content: space-between`.

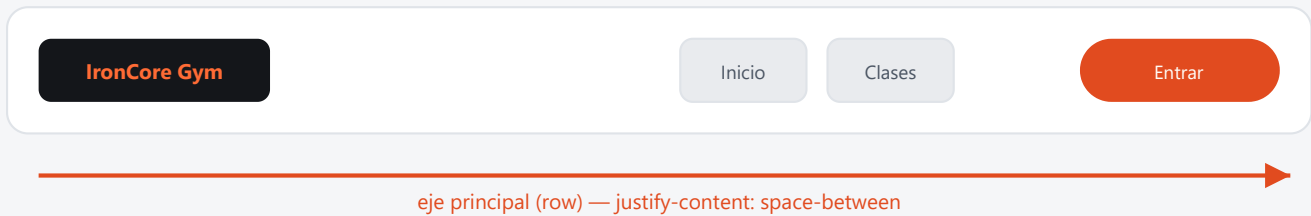


Fig. 3 — `display: flex` distribuye logo, nav y botón a lo largo del eje principal.

```
.site-header .container {
  display: flex;
  align-items: center;          /* centra verticalmente (eje cruzado) */
  justify-content: space-between; /* Logo a la izq., nav a la der. */
  gap: 1rem;
}

.site-nav ul {
  display: flex;
  align-items: center;
  gap: 1.75rem;
  list-style: none;
}
```

De `estilos.css`. El mismo `<header>` y `<nav>` se repiten en las 5 páginas.

6. CSS GRID — "SOBRE NOSOTROS" EN 2 COLUMNAS

La vista **Sobre nosotros** organiza su contenido principal (la historia + la imagen) en una cuadrícula de 2 columnas con `display: grid`, algo que Flexbox no resuelve tan bien cuando se necesita controlar el ancho exacto de cada columna.



Fig. 4 — `grid-template-columns: 1.1fr 0.9fr` reparte el espacio proporcionalmente.

```
.about-grid {
  display: grid;
  grid-template-columns: 1.1fr 0.9fr; /* 2 columnas */
  gap: 3rem;
  align-items: start;
}

/* En móvil, una sola columna: */
@media (max-width: 820px) {
  .about-grid { grid-template-columns: 1fr; }
}
```

```
<div class="about-grid">
  <article>
    <h2>Nuestra historia</h2>
    <p>...</p>
    <ul class="values-list">...</ul> // lista de valores
  </article>
  <figure>
    
    <figcaption>Nuestra sala principal...</figcaption>
  </figure>
</div>
```

De `sobre-nosotros.html`. Nota el `alt` descriptivo de la imagen y la lista de valores.

7. TARJETAS DE CLASES Y TRANSICIÓN EN HOVER

La vista **Servicios** repite 4 tarjetas (.card) usando display: grid con auto-fit, que acomoda automáticamente el número de columnas según el ancho disponible — sin media queries adicionales.

```
.card-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(240px, 1fr));
  gap: 1.75rem;
}

.card {
  transition: transform 0.3s ease, box-shadow 0.3s ease; /* suaviza el cambio */
}

.card:hover {
  transform: translateY(-8px); /* La tarjeta "flota" al pasar el cursor */
  box-shadow: var(--shadow);
}
```

CÓMO FUNCIONA LA TRANSICIÓN

Sin transition, el cambio de transform y box-shadow ocurriría de golpe. Al declarar transition: transform 0.3s ease, el navegador interpola 0.3 segundos de fotogramas intermedios entre el estado normal y el estado :hover, logrando el efecto "suave" que pide la consigna.

```
<article class="card">
  
  <div class="card-body">
    <h3>Musculación</h3>
    <p>Rutinas de fuerza con seguimiento personalizado...</p>
    <span class="price-tag">Incluido en todos los planes</span>
  </div>
</article>
```

De `servicios.html` : imagen + título + descripción, repetido 4 veces.

TABLA COMPARATIVA DE PLANES (REQUISITO OPCIONAL)

```
<table class="compare-table">
  <caption>Comparativa de planes mensuales</caption>
  <thead><tr><th>Plan</th><th>Sala de pesas</th><th>Clases</th><th>Precio</th></tr></thead>
  <tbody>
```

```
<tr><th scope="row">Pro</th><td>Sí</td><td>Hasta 8/mes</td><td>$40/mes</td></tr>
</tbody>
</table>
```

8. ANIMACIÓN DE ENTRADA (HERO DE INICIO)

El bloque destacado de **Inicio** aparece con una animación: el título y el subtítulo entran con un `fadeInUp` (se desvanecen hacia adentro mientras suben), y la imagen con un `fadeIn` algo retrasado para dar sensación de secuencia.

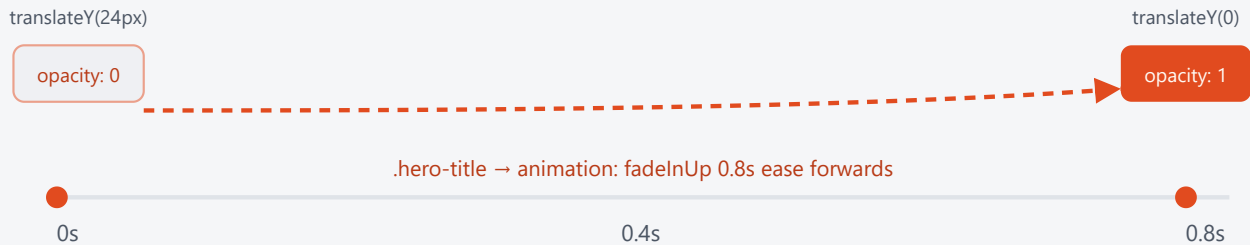


Fig. 5 — El navegador interpola entre el estado `from` y `to` del `@keyframes`.

```
@keyframes fadeInUp {  
  from { opacity: 0; transform: translateY(24px); }  
  to   { opacity: 1; transform: translateY(0); }  
}  
  
.hero-title {  
  opacity: 0; /* estado inicial antes de animar */  
  animation: fadeInUp 0.8s ease forwards;  
}  
  
.hero-subtitle {  
  opacity: 0;  
  animation: fadeInUp 0.8s ease 0.25s forwards; /* retraso → efecto escalonado */  
}
```

`forwards` hace que el elemento se quede en el estado final (`opacity: 1`) una vez termina la animación, en vez de volver al estado inicial.

9. FORMULARIOS ACCESIBLES Y VALIDACIÓN HTML5

El formulario de **Contacto** tiene 4 tipos de campo distintos (texto, email, select, textarea). Cada `<label>` está vinculado a su campo mediante `for/id` — así, al hacer clic en la etiqueta, el foco salta al campo (y un lector de pantalla anuncia la etiqueta correcta).

```
<label for="interes">Clase de interés</label>
<select id="interes" name="interes" required>
  <option value="" disabled selected>Selecciona una opción</option>
  <option value="musculacion">Musculación</option>
  ...
</select>

<label for="mensaje">Mensaje</label>
<textarea
  id="mensaje"
  name="mensaje"
  required
  minlength="10"           // exige al menos 10 caracteres
></textarea>
```

VALIDACIÓN NATIVA (SIN JAVASCRIPT)

Atributos como `required`, `type="email"` y `minlength` le piden al **propio navegador** que valide el campo antes de enviar el formulario: si falta un dato o el formato de correo no es válido, aparece un mensaje nativo (en el idioma del navegador) y el envío se detiene. Esto es "validación HTML5".

En **Iniciar sesión**, el mismo patrón `for/id` se usa en el checkbox "Recordarme":

```
<input type="checkbox" id="recordarme" name="recordarme" />
<label for="recordarme">Recordarme</label>
<a href="#">¿Olvidaste tu contraseña?</a>
```

10. EL MENÚ MÓVIL SIN JAVASCRIPT ("CHECKBOX HACK")

Como el proyecto no usa JavaScript, el menú hamburguesa de pantallas pequeñas se resuelve con un truco muy conocido en CSS: un `<input type="checkbox">` oculto que, al marcarse, cambia el estilo de sus *hermanos* en el HTML mediante el selector `~`.



Fig. 6 — El selector de hermano general (`~`) conecta el estado del checkbox con el estilo del nav.

```
<input type="checkbox" id="menu-toggle" class="menu-toggle-input" />
<label for="menu-toggle" class="nav-toggle">
  <span></span><span></span><span></span><span></span> // 3 líneas del icono
</label>
<nav class="site-nav">...</nav>
```

```
/* Oculto por defecto en móvil */
.site-nav { display: none; }

/* Al marcar el checkbox, el <nav> hermano pasa a mostrarse */
.menu-toggle-input:checked ~ .site-nav { display: flex; }

/* Las 3 líneas del icono se transforman en una "X" */
.menu-toggle-input:checked ~ .nav-toggle span:nth-child(1) { transform: translateY(7px) rotate(45deg); }
.menu-toggle-input:checked ~ .nav-toggle span:nth-child(2) { opacity: 0; }
```

IDEA CLAVE

Un `<label for="menu-toggle">` hace clic por ti sobre su `<input>` asociado. Como el checkbox tiene estado (`:checked` / sin marcar) y CSS puede "leer" ese estado con un pseudo-selector, se puede simular interactividad sin una sola línea de JavaScript.

11. TEMA CLARO/OSCURO CON VARIABLES CSS

Todos los colores del sitio están definidos como **variables CSS** (`--color-...`) en `:root`. El tema oscuro no duplica cada regla: solo redefine esas mismas variables cuando el sistema operativo del visitante tiene activado el modo oscuro (`prefers-color-scheme: dark`).

Claro · fondo #f5f6f8	Claro · texto #14161a	Primario #e14b1f	Oscuro · fondo #14161a	Oscuro · texto #f2f3f5
--	--	---	---	---

```
:root {
  --color-bg: #f5f6f8;      /* valores del tema claro */
  --color-text: #14161a;
}

@media (prefers-color-scheme: dark) {
  :root:not([data-theme="light"]) {
    --color-bg: #14161a;    /* se sobrescriben solo si el SO pide oscuro */
    --color-text: #f2f3f5;
  }
}

body { background-color: var(--color-bg); color: var(--color-text); }
```

CONTRASTE DE COLOR

En claro, el texto #14161a (casi negro) sobre fondo #f5f6f8 (gris muy claro) da un contraste muy alto. En oscuro, #f2f3f5 (blanco roto) sobre #14161a invierte la relación manteniendo el mismo contraste alto — así el texto se sigue leyendo bien en ambos temas, que es justo lo que pide la consigna.

12. DISEÑO RESPONSIVO

El layout cambia en dos puntos de quiebre (*breakpoints*), definidos con `@media` (`max-width: ...`):



Fig. 7 — Breakpoints usados en `estilos.css`.

Breakpoint	Qué cambia
<code>max-width: 820px</code>	El grid de "Sobre nosotros" pasa de 2 a 1 columna; el hero se apila verticalmente.
<code>max-width: 680px</code>	Aparece el botón hamburguesa; el <code><nav></code> pasa de fila a menú desplegable en columna.

Además, unidades como `clamp()` hacen que algunos tamaños de letra se ajusten de forma fluida sin necesidad de más media queries:

```
.hero-title { font-size: clamp(2.4rem, 5.5vw, 4rem); }  
/* nunca más chico que 2.4rem, nunca más grande que 4rem,  
   y en el medio escala con el ancho de la ventana (5.5vw) */
```

13. IMÁGENES PROPIAS DEL PROYECTO

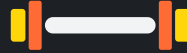
En vez de depender de fotos externas, todas las imágenes del sitio son ilustraciones **SVG** propias, guardadas en assets/. Al ser vectores, se ven nítidas en cualquier tamaño de pantalla y pesan muy poco.



hero-fitness.svg
Inicio



historia.svg
Sobre nosotros



musculacion.svg
Servicios



funcional.svg
Servicios



yoga.svg
Servicios



spinning.svg
Servicios



favicon.svg
Logo / pestaña

14. CHECKLIST DE REQUISITOS CUMPLIDOS

Requisito del PDF	¿Dónde está?	
Header + nav en Flexbox, en las 5 vistas	<code>estilos.css</code> → <code>.site-header</code> <code>.container</code> , <code>.site-nav</code>	✓
Un único <code>estilos.css</code> , sin estilos internos/en línea	Enlazado en el <code><head></code> de las 5 páginas	✓
Estructura semántica completa	<code>header</code> , <code>nav</code> , <code>main</code> , <code>footer</code> , <code>section</code> / <code>article</code>	✓
Metaetiquetas SEO	<code><head></code> de cada vista	✓
Diseño responsivo	<code>@media</code> en 680px y 820px + <code>clamp()</code>	✓
Tema claro/oscuro con buen contraste	<code>prefers-color-scheme</code> + variables CSS	✓
Inicio: sección destacada + animación de entrada + footer	<code>index.html</code> → <code>.hero</code> , <code>@keyframes</code> <code>fadeInUp</code>	✓
Sobre nosotros: imagen con <code>alt</code> , lista, Grid 2 columnas	<code>sobre-nosotros.html</code> → <code>.about-grid</code>	✓
Servicios: ≥3 tarjetas, hover suave, tabla opcional	<code>servicios.html</code> → <code>.card-grid</code> , <code>.compare-table</code>	✓
Contacto: formulario de 4 tipos de campo, <code>label for/id</code> , validación HTML5, dirección	<code>contacto.html</code>	✓
Iniciar sesión: usuario/contraseña, "Recordarme", enlaces	<code>iniciar-sesion.html</code>	✓

GLOSARIO RÁPIDO

- **Flexbox**: modelo de layout en una dimensión (fila o columna) para alinear y repartir espacio entre elementos.
- **CSS Grid**: modelo de layout en dos dimensiones (filas y columnas) para cuadrículas.
- **Especificidad**: regla que decide qué declaración CSS "gana" cuando dos selectores afectan al mismo elemento (un id pesa más que una clase, una clase más que una etiqueta). En `estilos.css` se usó `!important` en `.nav-cta` solo para garantizar que el botón "Iniciar sesión" mantenga su texto claro frente a la regla general de color de los enlaces del nav — una excepción puntual, no la norma.
- **Variable CSS (--nombre)**: valor reutilizable que se lee con `var(--nombre)` y puede cambiar según el contexto (aquí, según el tema).
- **Pseudo-clase**: selector que apunta a un estado del elemento, como `:hover` , `:focus` o `:checked`.

- **Media query** (`@media`): bloque de CSS que solo se aplica si se cumple una condición (ej. ancho de pantalla).